

SIMATS ENGINEERING
ASSESSMENT TOOL 1
Experiments

COURSE NAME: Computer Networks

COURSE CODE: CSA07

COURSE OUTCOME COVERED

CO3: Analyze the performance of core network protocols such as TCP, UDP, HTTP, and DNS under varying conditions

Assessment Tool Weightage: 40%

Code of Conduct:

(192492056)

I V. Shreshika (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student V. Shreshika

Experiments

Assessment 1: TCP Three-Way Handshake in Wireshark

Aim

To capture TCP packets during connection setup and identify the **SYN**, **SYN-ACK**, and **ACK** packets.

Procedure

1. Open Wireshark and select the active **Wi-Fi** interface.
2. Start packet capture.
3. Open Command Prompt and execute:

```
curl https://example.com
```

4. Stop the capture.
5. Apply the filter:

```
tcp.flags.syn == 1
```

6. Identify the **SYN** and **SYN-ACK** packets.
7. Select the corresponding TCP conversation to identify the final **ACK** packet.
8. Observe the TCP flags in the packet details.

Output / Observation

Packet	TCP Flag	Purpose
1	SYN	Client requests a TCP connection
2	SYN, ACK	Server accepts the request and sends its own SYN
3	ACK	Client acknowledges the server

Result

Wi-Fi

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

tcpstream eq 27

No.	Time	Source	Destination	Protocol	Length	Info
428	41.099514000	2006:40f4:40d4:b73a	2006:4700:83b2:c468	TCP	88	62204 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1440 WS=256 SACK_PERM
429	41.159460900	2006:4700:83b2:c468	2006:40f4:40d4:b73a	TCP	88	80 → 62204 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1300 SACK_PERM WS=8192
430	41.159471900	2006:40f4:40d4:b73a	2006:4700:83b2:c468	TCP	74	62204 → 80 [ACK] Seq=1 Ack=1 Win=65280 Len=0
431	41.159471900	2006:40f4:40d4:b73a	2006:4700:83b2:c468	HTTP	306	GET /WP/WTZBWP6wS7A8gUgPCGqUABBRTEU9ufqVGHjWZYNQDmVR5ngQUBEKI:0hQYc4q8p74K1P8hupLQCEBc
432	41.194516900	2006:4700:83b2:c468	2006:40f4:40d4:b73a	TCP	74	80 → 62204 [ACK] Seq=1 Ack=233 Win=131072 Len=0
433	41.238406300	2006:4700:83b2:c468	2006:40f4:40d4:b73a	TCP	1374	80 → 62204 [ACK] Seq=1 Ack=233 Win=131072 Len=1300 [TCP PDU reassembled in 434]
434	41.238406300	2006:4700:83b2:c468	2006:40f4:40d4:b73a	OCSP	1049	Response
436	41.238512900	2006:40f4:40d4:b73a	2006:4700:83b2:c468	TCP	88	62204 → 80 [ACK] Seq=233 Ack=2276 Win=65280 Len=0 SLE=1301 SRE=2276

Frame 428: Packet, 88 bytes on wire (688 bits), 88 bytes captured (688 bits) on interface
 Ethernet II, Src: Intel_F7:1b:15 (08:0f:65:f7:1b:15), Dst: d6:b6:5e:24:33:12 (d6:b6:5e:24:33:12)
 Internet Protocol Version 6, Src: 2006:40f4:40d4:b73a, Dst: 2006:4700:83b2:c468
 Transmission Control Protocol, Src Port: 62204, Dst Port: 80, Seq: 0, Len: 0

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

tcpstream eq 27

No.	Time	Source	Destination	Protocol	Length	Info
428	41.099514000	2006:40f4:40d4:b73a	2006:4700:83b2:c468	TCP	88	62204 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1440 WS=256 SACK_PERM
429	41.159460900	2006:4700:83b2:c468	2006:40f4:40d4:b73a	TCP	88	80 → 62204 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1300 SACK_PERM WS=8192
430	41.159471900	2006:40f4:40d4:b73a	2006:4700:83b2:c468	TCP	74	62204 → 80 [ACK] Seq=1 Ack=1 Win=65280 Len=0
431	41.159471900	2006:40f4:40d4:b73a	2006:4700:83b2:c468	HTTP	306	GET /WP/WTZBWP6wS7A8gUgPCGqUABBRTEU9ufqVGHjWZYNQDmVR5ngQUBEKI:0hQYc4q8p74K1P8hupLQCEBc
432	41.194516900	2006:4700:83b2:c468	2006:40f4:40d4:b73a	TCP	74	80 → 62204 [ACK] Seq=1 Ack=233 Win=131072 Len=0
433	41.238406300	2006:4700:83b2:c468	2006:40f4:40d4:b73a	TCP	1374	80 → 62204 [ACK] Seq=1 Ack=233 Win=131072 Len=1300 [TCP PDU reassembled in 434]
434	41.238406300	2006:4700:83b2:c468	2006:40f4:40d4:b73a	OCSP	1049	Response
436	41.238512900	2006:40f4:40d4:b73a	2006:4700:83b2:c468	TCP	88	62204 → 80 [ACK] Seq=233 Ack=2276 Win=65280 Len=0 SLE=1301 SRE=2276

Frame 429: Packet, 86 bytes on wire (688 bits), 86 bytes captured (688 bits) on interface
 Ethernet II, Src: d6:b6:5e:24:33:12 (d6:b6:5e:24:33:12), Dst: Intel_F7:1b:15 (08:0f:65:f7:1b:15)
 Internet Protocol Version 6, Src: 2006:4700:83b2:c468, Dst: 2006:40f4:40d4:b73a
 Transmission Control Protocol, Src Port: 80, Dst Port: 62204, Seq: 0, ACK: 1, Len: 0

Wi-Fi

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

tcpstream eq 27

No.	Time	Source	Destination	Protocol	Length	Info
428	41.099514000	2006:40f4:40d4:b73a	2006:4700:83b2:c468	TCP	88	62204 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1440 WS=256 SACK_PERM
429	41.159460900	2006:4700:83b2:c468	2006:40f4:40d4:b73a	TCP	88	80 → 62204 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1300 SACK_PERM WS=8192
430	41.159471900	2006:40f4:40d4:b73a	2006:4700:83b2:c468	TCP	74	62204 → 80 [ACK] Seq=1 Ack=1 Win=65280 Len=0
431	41.159471900	2006:40f4:40d4:b73a	2006:4700:83b2:c468	HTTP	306	GET /WP/WTZBWP6wS7A8gUgPCGqUABBRTEU9ufqVGHjWZYNQDmVR5ngQUBEKI:0hQYc4q8p74K1P8hupLQCEBc
432	41.194516900	2006:4700:83b2:c468	2006:40f4:40d4:b73a	TCP	74	80 → 62204 [ACK] Seq=1 Ack=233 Win=131072 Len=0
433	41.238406300	2006:4700:83b2:c468	2006:40f4:40d4:b73a	TCP	1374	80 → 62204 [ACK] Seq=1 Ack=233 Win=131072 Len=1300 [TCP PDU reassembled in 434]
434	41.238406300	2006:4700:83b2:c468	2006:40f4:40d4:b73a	OCSP	1049	Response
436	41.238512900	2006:40f4:40d4:b73a	2006:4700:83b2:c468	TCP	88	62204 → 80 [ACK] Seq=233 Ack=2276 Win=65280 Len=0 SLE=1301 SRE=2276

Frame 430: Packet, 74 bytes on wire (592 bits), 74 bytes captured (592 bits) on interface
 Ethernet II, Src: Intel_F7:1b:15 (08:0f:65:f7:1b:15), Dst: d6:b6:5e:24:33:12 (d6:b6:5e:24:33:12)
 Internet Protocol Version 6, Src: 2006:40f4:40d4:b73a, Dst: 2006:4700:83b2:c468
 Transmission Control Protocol, Src Port: 62204, Dst Port: 80, Seq: 1, Ack: 1, Len: 0

Assessment 2: DNS Query Using UDP in Wireshark

Aim

To capture a DNS query using UDP in Wireshark and compare the UDP header with the TCP header.

Procedure

1. Open Wireshark and select Wi-Fi.
2. Start packet capture.
3. Open Command Prompt.
4. Enter:

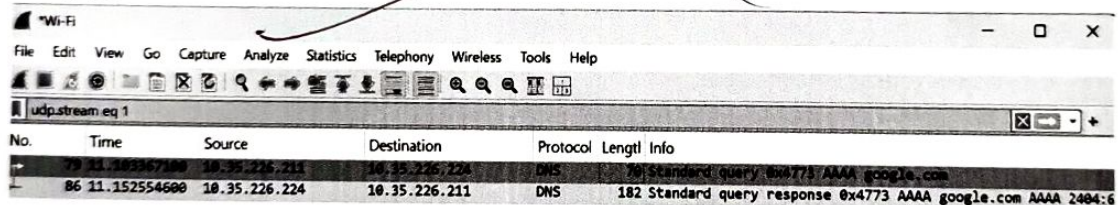
```
nslookup google.com
```

5. Stop the capture in Wireshark.
6. Apply the filter:

```
dns && udp
```

7. Select the DNS query packet.
8. Expand User Datagram Protocol and observe the UDP header.

Output:



No.	Time	Source	Destination	Protocol	Length	Info
79	11.152554600	10.35.226.211	10.35.226.224	DNS	70	Standard query 0x4773 AAAA google.com
86	11.152554600	10.35.226.224	10.35.226.211	DNS	182	Standard query response 0x4773 AAAA google.com AAAA 2404:6

```
> Frame 79: Packet, 70 bytes on wire (560 bits), 70 bytes captured (560 bits) on interface 0
> Ethernet II, Src: Intel_f7:1b:15 (98:ef:65:f7:1b:15), Dst: c2:a8:97:a5:fa:74:98:af
> Internet Protocol Version 4, Src: 10.35.226.211, Dst: 10.35.226.224
> User Datagram Protocol, Src Port: 63344, Dst Port: 53
> Domain Name System (query)
    0000 c2 a8 97 a5 fa 74 98 af 65 f7 1b 15 00 00 45 00 .....t..e
    0010 00 38 01 20 00 00 00 11 00 00 0a 23 e2 d3 0a 23 ..8.....:
    0020 e2 e0 f7 70 00 35 00 24 da 2f 47 73 01 00 00 01 ...p:5-$..
    0030 00 00 00 00 00 00 06 67 6f 6f 67 6c 65 03 63 6f .....g..o
    0040 6d 00 00 1c 00 01                                     m.....
```

Assessment 3: TCP Retransmission vs UDP

Aim

To simulate packet loss and observe how TCP retransmits lost packets, while UDP does not retransmit them.

Procedure

Part A — Capture TCP retransmission

1. Open Wireshark.
2. Select Wi-Fi and start capturing.
3. Open Command Prompt.
4. Generate TCP traffic:

```
curl https://example.com
```

5. While traffic is being generated, briefly turn Wi-Fi OFF and ON.
6. Generate the traffic again if necessary.
7. Stop Wireshark.
8. Apply this filter:

```
tcp.analysis.retransmission
```

Wireshark may display:

[TCP Retransmission]

This indicates that TCP retransmitted a packet.

Part B — Observe UDP

Generate DNS traffic:

```
nslookup google.com
```

Use the filter:

```
dns && udp
```

You will see:

DNS Query

DNS Response

UDP itself does not show a TCP-style retransmission when a packet is lost.

```

Frame 163: Packet, 256 bytes on wire (2048 bits), 256 bytes captured (2048 bits) on 14
Ethernet II, Src: c2:a8:97:a5:fa:7a (c2:a8:97:a5:fa:7a), Dst: Intel_77:1b:15 (98:af:85
882:1Q Virtual LAN, PVID: 0, DEI: 0, ID: 0
Internet Protocol Version 6, Src: 2001:4800:48a7:400::, Dst: 2000:40f4:4800:c3a6:bcb6:
Transmission Control Protocol, Src Port: 443, Dst Port: 50066, Seq: 248, Ack: 143, Len

```

The screenshot shows the Wireshark network protocol analyzer interface. The top menu bar includes File, Edit, View, Go, Capture, Analyze, Statistics, Telephony, Wireless, Tools, and Help. Below the menu is a toolbar with various icons for packet capture and analysis. The main display area is divided into three panes:

- Packet List Pane:** Displays a list of captured packets. The columns are No., Time, Source, Destination, Protocol, and Length. The packets shown are:
 - No. 116, Time 12.673328000, Source 10.35.226.211, Destination 10.35.226.224, Protocol DNS, Length 86. Standard query 0x0001 PTR 224.226.35.10.in-addr.arpa
 - No. 117, Time 12.677573600, Source 10.35.226.224, Destination 10.35.226.211, Protocol DNS, Length 86. Standard query response 0x0001 No such name PTR 224.226.35.10.in-addr.arpa
 - No. 118, Time 12.680719500, Source 10.35.226.211, Destination 10.35.226.224, Protocol DNS, Length 70. Standard query 0x0002 A google.com
 - No. 119, Time 12.686238000, Source 10.35.226.224, Destination 10.35.226.211, Protocol DNS, Length 166. Standard query response 0x0002 A google.com A 192.178.158.130 A 192.178.158.139 A 192.178.158.133
 - No. 120, Time 12.693581300, Source 10.35.226.211, Destination 10.35.226.224, Protocol DNS, Length 70. Standard query 0x0003 AAAA google.com
 - No. 121, Time 12.930851400, Source 10.35.226.224, Destination 10.35.226.211, Protocol DNS, Length 182. Standard query response 0x0003 AAAA google.com AAAA 2004:6800:4013:813::65 AAAA 2004:6800:4013:813::65
- Packet Details Pane:** Shows the hierarchical structure of the selected packet (No. 121). It includes:
 - Standard query response 0x0003 AAAA google.com AAAA 2004:6800:4013:813::65 AAAA 2004:6800:4013:813::65
 - Header
 - Question
 - Answer
 - Authority
 - Additional
- Packet Bytes Pane:** Shows the raw data of the selected packet in hexadecimal and ASCII format.

```

> Frame 116: Packet, 86 bytes on wire (688 bits), 86 bytes captured (688 bits) on interface 0
> Ethernet II, Src: Intel-7f:3b:15 (08:0f:65:f7:3b:15), Dst: c2:a8:97:a5:fa:74 (c2:a8:97:a5:fa:74)
> Internet Protocol Version 4, Src: 10.35.226.211, Dst: 10.35.226.224
> User Datagram Protocol, Src Port: 59098, Dst Port: 53
> Domain Name System (query)
0000 c2 a8 97 a5 fa 74 98 af 65 f7 1b 15 08 00 45 00  ...c...E-
0010 00 48 01 b2 00 00 00 11 00 00 04 23 62 03 0a 23  ...H.....-
0020 62 c0 cf 6a 00 35 34 0a 32 34 03 32 36 02 33    ...5-4-7-...
0030 35 02 31 30 00 00 00 63 32 34 03 32 36 02 33    ...2 24-226-3
0040 35 02 31 30 00 00 00 6a 2a 61 64 72 04 61 72 70  5-10-in-addr-arp
0050 61 00 00 0c 00 01                                a-...

```

Aim

To capture DNS traffic, identify A, AAAA and MX query types, observe the response, and measure DNS resolution time.

Procedure

1. Open Wireshark.
2. Select Wi-Fi and click Start Capture.
3. Open Command Prompt.
4. Run:

nslookup google.com

5. Stop the capture.
6. Apply the filter:

dns

Step 1: Identify A Record

In CMD:

nslookup -type=A google.com

In Wireshark, look for:

Standard query A google.com

The response will contain an IPv4 address, for example:

google.com → 142.x.x.x

A = IPv4 address

Step 2: Identify AAAA Record

Run:

nslookup -type=AAAA google.com

Wireshark should show:

Standard query AAAA google.com

Standard query response AAAA google.com

AAAA = IPv6 address

Your previous screenshot already showed this successfully:

Standard query 0x0003 AAAA google.com

Standard query response 0x0003 AAAA google.com
So you already have a good AAAA output.

Step 3: Identify MX Record

Run:

nslookup -type=MX google.com

Wireshark should show:

Standard query MX google.com
Standard query response MX google.com

MX = Mail Exchange record, which identifies mail servers for the domain.

Step 4: Measure Resolution Time

In Wireshark, find:

DNS Query

and its matching:

DNS Response

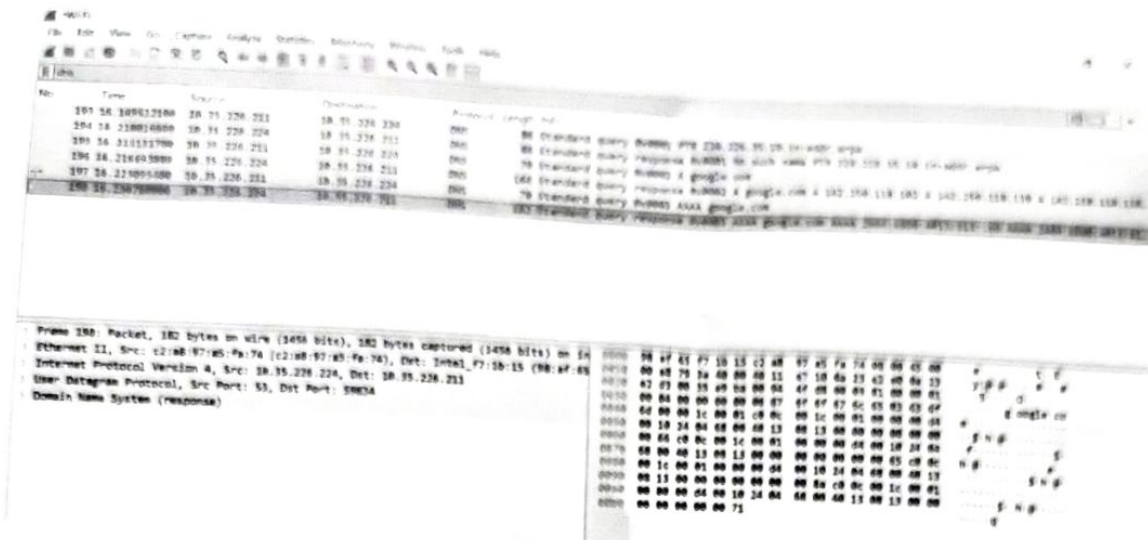
For example:

Query 12.893301 sec
Response 12.930511 sec

Therefore:

Resolution time = 12.930511 - 12.893301
= 0.037210 seconds
= 37.21 ms

Your actual time will be different.



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation